

Lucas-Threshold Localization for Semiprime Factorization: Safe Intervals, Binary Hit/Miss Probes, and Fast Modular Product Evaluation

IamLupo

13 September 2026

Abstract

This note records the continuation of an experimental investigation into semiprime factor localization using binomial coefficients and Lucas' theorem. The central construction converts a factor test into a binary hit/miss predicate inside a suitable interval of candidate primes. The work developed from a quotient-one Lucas regime, through interval subdivision, to an implementation that performs only one greatest-common-divisor operation per binary probe. Several computational optimizations were then explored, including cumulative numerator products, paired factors, and Montgomery multiplication. The present note records the mathematical constructions, algorithmic steps, and experimental outputs developed after the preceding manuscript. No statistical or performance interpretation is attempted here; the numerical results are reported as experimental observations only.

Contents

1	Scope and Continuation	2
2	Reduction to a Binomial Divisibility Test	2
3	Lucas' Theorem and the Quotient-One Regime	3
4	Binary Hit/Miss Localization	4
4.1	Why the quotient condition matters	5
5	Safe Segmentation	5
6	Constructing the Initial Interval from N Alone	5
7	Worked Small Example	6
8	Experiments 138–145: Localization and Safe Intervals	7
8.1	Experiment 138	7
8.2	Experiments 139–140	7
8.3	Experiments 141–142	7
8.4	Experiments 143–144	7
8.5	Experiment 145	7
9	Large-Scale Implementation	8
9.1	Experiment 146	8
9.2	Experiment 147	8
9.3	Experiment 148	8

10 Experiments 149–152: C++ and Block GCD	8
10.1 Experiment 149	8
10.2 Experiment 150	8
10.3 Experiment 151	8
10.4 Experiment 152	8
11 Experiments 153–154: One GCD per Binary Probe	9
12 Experiment 155: Pairing Numerator Terms	9
13 Experiment 156: Montgomery Multiplication	10
13.1 Montgomery representation	10
13.2 Experiment configuration	10
14 Computational Procedure in Pseudocode	11
15 Alternative View: Removing the Segment Layer	11
16 Current Research State	12
17 Recorded Experimental Results	12
18 Conclusion	13

1 Scope and Continuation

The preceding research established an arithmetic-sequence construction whose special repeated-coefficient form leads to binomial coefficients modulo prime divisors of a semiprime. The present note begins from the resulting Lucas-theoretic factor-localization mechanism and follows its subsequent development.

The main objects are a semiprime

$$N = pq, \quad p \leq q,$$

and an interval containing the smaller prime factor p . The objective is not to compute a general binomial coefficient as a large integer, but to construct probes whose divisibility by p changes at a controllable threshold.

The work reported here contains the following successive ideas:

1. reduction of the relevant arithmetic-sequence quantity to a binomial coefficient;
2. use of Lucas' theorem in a quotient-one regime;
3. construction of a binary hit/miss predicate;
4. construction of safe local intervals satisfying $U < 2L$;
5. application of the construction starting from an interval derived from N alone;
6. elimination of per-term GCD checks;
7. reuse of a cumulative numerator product during binary search;
8. paired numerator multiplication;
9. Montgomery multiplication for the modular product.

The experiments below are recorded as experiments rather than as a claim of a complete practical factoring algorithm.

2 Reduction to a Binomial Divisibility Test

Consider the special repeated coefficient pattern used in the arithmetic-sequence construction. Let the repeated parameter be z , and let the resulting polynomial degree be

$$d = 2z - 1.$$

For a prime divisor $r \mid N$, the construction gives the congruence

$$v_k(z) \equiv -\binom{k-2}{d} \pmod{r}.$$

Consequently,

$$\gcd(v_k(z), N) = \gcd\left(\binom{k-2}{d}, N\right).$$

Writing

$$m = k - 2,$$

the relevant quantity becomes

$$\gcd\left(\binom{m}{d}, N\right).$$

When

$$d < p,$$

we have

$$\gcd(d!, N) = 1,$$

provided $N = pq$ with p the smaller prime factor. Therefore, for divisibility by p or q , the denominator $d!$ may be inverted modulo the relevant prime, and the numerator form can be used:

$$\binom{m}{d} = \frac{m(m-1) \cdots (m-d+1)}{d!}.$$

Thus

$$p \mid \binom{m}{d} \iff p \mid \prod_{j=m-d+1}^m j$$

under the condition $d < p$.

Equivalently, defining

$$P(m, d) = \prod_{j=m-d+1}^m j,$$

we may test

$$\gcd(P(m, d), N).$$

The later implementation evaluates this product modulo N and performs the GCD only at the end of a probe.

3 Lucas' Theorem and the Quotient-One Regime

Let

$$N = pq, \quad p < q,$$

and consider a binomial coefficient

$$\binom{m}{d}.$$

Lucas' theorem states that the residue modulo a prime p is determined by the base- p digits of m and d .

For the construction used here, the relevant simplification occurs when

$$p \leq m < 2p.$$

In this range,

$$m = p + r, \quad 0 \leq r < p.$$

If moreover

$$d < p,$$

then Lucas' theorem gives

$$\binom{m}{d} \equiv \binom{1}{0} \binom{r}{d} \pmod{p}.$$

Hence

$$p \mid \binom{m}{d} \iff d > r.$$

Since

$$r = m - p,$$

we obtain

$$p \mid \binom{m}{d} \iff p > m - d.$$

Introducing the threshold

$$t = m - d,$$

we obtain the central predicate

$$\boxed{p \mid \binom{m}{m-t} \iff p > t} \tag{1}$$

provided

$$p \leq m < 2p, \quad m - t < p.$$

The equivalent numerical condition for a whole candidate interval

$$p \in [L, U]$$

when choosing $m = U$ is

$$U < 2L. \tag{2}$$

Under (2), every candidate $p \in [L, U]$ lies in the quotient-one regime with respect to $m = U$.

4 Binary Hit/Miss Localization

Suppose a safe interval

$$[L, U]$$

satisfies

$$U < 2L.$$

Choose a midpoint

$$t = \left\lfloor \frac{L + U}{2} \right\rfloor,$$

and fix

$$m = U, \quad d = U - t.$$

Then the Lucas predicate becomes

$$p \mid \binom{U}{U-t} \iff p > t.$$

Therefore a single probe gives the binary decision

$$\begin{cases} \text{HIT}, & p > t, \\ \text{MISS}, & p \leq t. \end{cases}$$

The interval update is consequently

$$\begin{cases} [L, t], & \text{MISS}, \\ [t + 1, U], & \text{HIT}. \end{cases}$$

This is a standard binary localization step, with the Lucas construction providing the predicate.

4.1 Why the quotient condition matters

If the interval is too wide, the value

$$\left\lfloor \frac{m}{p} \right\rfloor$$

need not be constant throughout the candidate interval. For example, with $m = 14$ and $p \in [4, 14]$, the quotient takes multiple values:

$$\left\lfloor \frac{14}{4} \right\rfloor = 3, \quad \left\lfloor \frac{14}{7} \right\rfloor = 2, \quad \left\lfloor \frac{14}{8} \right\rfloor = 1.$$

The simple threshold derivation leading to (1) therefore does not apply uniformly over $[4, 14]$.

This is why the experiments introduced local safe segments.

5 Safe Segmentation

A convenient sufficient rule is

$$U \leq \alpha L, \quad \alpha < 2.$$

The experiments used

$$\alpha = 1.5.$$

Then automatically

$$U < 2L,$$

so the quotient-one Lucas regime is valid throughout every segment.

For an interval $[L_0, R_0]$, segments were constructed iteratively by taking

$$U = \lfloor 1.5L \rfloor$$

until the upper end of the original interval was reached.

For example, the interval

$$[4, 14]$$

can be decomposed as

$$[4, 6], \quad [7, 10], \quad [11, 14],$$

where each segment satisfies $U < 2L$.

Within a segment, binary search uses the Lucas threshold. The outer iteration determines which safe segment contains the factor.

6 Constructing the Initial Interval from N Alone

Suppose

$$q/p \leq C$$

for a chosen search assumption C . Since

$$N = pq,$$

we have

$$N = pq \leq Cp^2,$$

so

$$p \geq \sqrt{N/C}.$$

Together with

$$p \leq \sqrt{N},$$

we obtain the N -only interval

$$\left[\left\lceil \sqrt{N/C} \right\rceil, \left\lfloor \sqrt{N} \right\rfloor \right]. \quad (3)$$

The experiments used a broad assumed bound such as

$$C = 16,$$

followed by safe-segment subdivision.

No prime factor is supplied to the procedure when constructing this initial interval.

7 Worked Small Example

A small example illustrates the actual probe mechanics. Consider

$$N = 221 = 13 \cdot 17.$$

Assume only $N = 221$ is known and take $C = 16$.

The initial lower and upper bounds are

$$L_0 = \left\lceil \sqrt{221/16} \right\rceil = 4,$$

$$R_0 = \left\lfloor \sqrt{221} \right\rfloor = 14.$$

Using the 1.5 subdivision rule gives

$$[4, 6], \quad [7, 10], \quad [11, 14].$$

The factor 13 lies in the final segment.

For

$$[L, U] = [11, 14],$$

choose

$$t = \left\lfloor \frac{11 + 14}{2} \right\rfloor = 12, \quad m = 14, \quad d = 14 - 12 = 2.$$

Then

$$\binom{14}{2} = 91.$$

Since

$$221 = 13 \cdot 17, \quad 91 = 7 \cdot 13,$$

we have

$$\gcd(91, 221) = 13.$$

Thus the probe is a HIT and identifies the factor

$$\boxed{13}.$$

Because $d = 2 < 13$, the denominator may be ignored for divisibility purposes. The implementation can instead evaluate the numerator

$$14 \cdot 13 = 182,$$

and obtain

$$\gcd(182, 221) = 13.$$

The sequence of logical operations is therefore

$$[11, 14] \longrightarrow t = 12 \longrightarrow \gcd\left(\prod_{j=13}^{14} j, 221\right) = 13 \longrightarrow p > 12.$$

8 Experiments 138–145: Localization and Safe Intervals

The experiments in this group developed and tested the threshold construction before the large-scale C++ implementation.

8.1 Experiment 138

A safe interval $[L, R]$ was used with

$$m = R, \quad t = \left\lfloor \frac{L + R}{2} \right\rfloor, \quad d = R - t.$$

Under the safe assumptions, the resulting probe was interpreted as

$$\text{HIT} \iff p > t.$$

The experiment tested this on small semiprimes.

8.2 Experiments 139–140

The midpoint construction was further examined, including hidden factor positions rather than placing the true factor at a predetermined midpoint. The threshold-based left/right interpretation was retained.

8.3 Experiments 141–142

The initial interval was constructed from N alone using

$$L = \left\lceil \sqrt{N/C} \right\rceil, \quad R = \left\lfloor \sqrt{N} \right\rfloor.$$

The experiments varied the assumed ratio bound C and examined the effect of wide initial intervals. This motivated an explicit distinction between an arbitrary initial interval and a quotient-consistent safe interval.

8.4 Experiments 143–144

These experiments attempted a fixed quotient window over a broad candidate interval. A general condition for a quotient- a regime was identified:

$$aR \leq m < (a + 1)L. \tag{4}$$

A nonempty window requires

$$aR < (a + 1)L. \tag{5}$$

Very wide intervals need not admit such a common window.

8.5 Experiment 145

The broad interval was therefore subdivided into local segments satisfying

$$U \leq 1.5L.$$

Binary Lucas probes were then performed inside each segment. The experiment tested the resulting N -only factorization procedure over multiple small semiprime ratio ranges.

9 Large-Scale Implementation

The next series of experiments moved the probe calculation from Python to C++. The main implementation problem became the evaluation of

$$P(a, b) = \prod_{x=a}^b x \pmod{N}$$

and the frequency of GCD calculations.

9.1 Experiment 146

Direct exact numerator-product and GCD evaluation was tested at approximately

$$N \sim 10^{10}, \quad 10^{12}, \quad 10^{14}, \quad 10^{16}, \quad 10^{18}.$$

The larger cases motivated the move away from direct Python big-integer products.

9.2 Experiment 147

Block-product variants were tested in Python. Several block sizes were considered in an attempt to reduce repeated GCD work.

9.3 Experiment 148

The modular product itself was microbenchmarked using several strategies, including a direct modular loop, tree-based multiplication, and chunked products. This motivated the use of C++ for the hot loop.

10 Experiments 149–152: C++ and Block GCD

10.1 Experiment 149

A C++ implementation processed an approximately 10^{18} semiprime by computing the numerator product modulo N and using block-level GCD checks.

10.2 Experiment 150

The raw modular multiplication loop was benchmarked on a numerator interval of approximately 5.2×10^7 terms. Different block sizes were compared with direct modular multiplication.

10.3 Experiment 151

A full C++ implementation initially performed a GCD after every numerator term. Two 10^{18} -scale cases were successfully factored, but the GCD frequency was extremely high.

10.4 Experiment 152

Block GCD was introduced. A block size of 256 was used, replacing a GCD after every term by one GCD after each block. The experiment included:

- two cases near 10^{18} ;
- one case near 10^{19} .

All reported cases were factored correctly. The implementation reduced the number of GCD calls by approximately the block-size factor.

11 Experiments 153–154: One GCD per Binary Probe

Experiment 153 removed block-level GCD checks entirely. For each binary probe, the complete numerator product was computed modulo N , followed by one GCD:

$$G = \gcd(P(a, b), N).$$

The binary search therefore used approximately one GCD per probe rather than one GCD per numerator term or block.

For the reported 10^{18} cases, the experiment used approximately 85–88 probes and the same number of GCDs. For the reported 10^{19} case, 103 probes and 103 GCDs were recorded.

Experiment 154 fixed

$$m = U$$

within each safe segment throughout the binary search. If a probe moved the threshold to the left, the numerator product could be extended cumulatively rather than recomputed from scratch.

The cumulative product has the form

$$P(t) = \prod_{j=t+1}^U j.$$

If a later threshold satisfies $t_2 < t_1$, then

$$P(t_2) = P(t_1) \prod_{j=t_2+1}^{t_1} j.$$

Thus only newly introduced factors need to be multiplied.

12 Experiment 155: Pairing Numerator Terms

The next implementation optimization paired consecutive numerator factors. Instead of reducing after every individual factor,

$$A \leftarrow A \cdot x \pmod{N}, \quad A \leftarrow A \cdot y \pmod{N},$$

the implementation formed

$$xy$$

and updated

$$A \leftarrow A \cdot (xy) \pmod{N}.$$

For the factor ranges under consideration,

$$x, y \leq \lfloor \sqrt{N} \rfloor$$

and adjacent unequal terms satisfy

$$xy < N.$$

Thus the pair itself fits as a residue modulo N without a separate reduction.

The experiment compared the paired implementation with the scalar implementation on cases near 10^{18} and 10^{19} . Both versions returned the same factor and the same number of binary probes and GCD calls.

13 Experiment 156: Montgomery Multiplication

The next implementation replaced the general modular reduction in the paired-product loop by Montgomery multiplication.

13.1 Montgomery representation

For an odd modulus N , let

$$R = 2^{64}, \quad \gcd(R, N) = 1.$$

An integer x is represented in Montgomery form as

$$\bar{x} = xR \pmod{N}.$$

Montgomery multiplication computes

$$\text{MontMul}(\bar{x}, \bar{y}) \equiv xyR \pmod{N},$$

without performing an ordinary division by N .

For a 64-bit modulus, one convenient reduction is based on

$$n' \equiv -N^{-1} \pmod{2^{64}}.$$

For

$$t = xy,$$

set

$$m = (t \bmod 2^{64})n' \bmod 2^{64},$$

and then

$$u = \frac{t + mN}{2^{64}}.$$

If $u \geq N$, subtract N . The resulting value is the Montgomery product.

The implementation included a self-test comparing Montgomery multiplication against ordinary modular multiplication on random pairs before performing the factorization experiments.

13.2 Experiment configuration

The reported cases included two semiprimes near 10^{18} and one near 10^{19} . The implementation compared:

1. paired multiplication using ordinary 128-bit modular reduction;
2. paired multiplication using Montgomery reduction.

The reported runs gave:

$$\text{montgomery_selftest} = 1$$

for every case, and both implementations returned the same candidate factors, probe counts, product-step counts, and GCD counts.

The reported 10^{18} cases used 87 and 86 probes respectively, while the reported 10^{19} case used 90 probes. Each probe corresponded to exactly one GCD.

14 Computational Procedure in Pseudocode

The current procedure may be represented as follows.

```

Input: N
Choose assumed ratio bound C

L0 = ceil(sqrt(N / C))
R0 = floor(sqrt(N))

Construct safe segments [L,U] satisfying U < 2L

For each segment [L,U]:
    m = U
    left = L
    right = U
    accumulator = 1

    while left < right:
        t = floor((left + right) / 2)
        d = m - t

        Extend accumulator to obtain
            product(t+1, ..., m) mod N

        g = gcd(accumulator, N)

        if g is a nontrivial factor:
            return g

        # Lucas threshold gives MISS:
        right = t

Return failure if no segment produces a factor

```

For the HIT case, the factor is returned directly by the GCD. In the MISS case, the search proceeds toward the lower half of the current segment according to the threshold predicate.

15 Alternative View: Removing the Segment Layer

The segmentation requirement arises from the use of the simple quotient-one form of Lucas' theorem. For a general choice of m and a candidate prime p , write

$$m = ap + r, \quad 0 \leq r < p.$$

Then the base- p representation contains the quotient digit a . The clean predicate

$$p > t$$

is specific to the quotient-one regime together with the chosen relation

$$d = m - t.$$

For a broad interval such as

$$[4, 14],$$

the quantity

$$\left\lfloor \frac{14}{p} \right\rfloor$$

changes with p . Therefore the current derivation does not yield one global threshold predicate for the whole interval.

This motivates a further mathematical direction: characterize the Lucas predicate across multiple quotient regimes and determine whether the candidate interval can be traversed directly, without first partitioning it into quotient-one segments.

In particular, the desired form would replace explicit product evaluation by an arithmetic predicate on a candidate range, or by a direct description of the subsets of the interval on which the Lucas residue vanishes.

16 Current Research State

The work recorded here has established the following chain of constructions:

$$\begin{aligned} \text{arithmetic-sequence construction} &\longrightarrow \binom{m}{d} \longrightarrow \text{Lucas criterion} \\ &\longrightarrow \text{threshold predicate} \longrightarrow \text{binary localization.} \end{aligned}$$

The computational implementation has successively moved through

$$\begin{aligned} \text{termwise GCD} &\longrightarrow \text{block GCD} \longrightarrow \text{one GCD/probe} \\ &\longrightarrow \text{cumulative products} \longrightarrow \text{paired products} \longrightarrow \text{Montgomery multiplication.} \end{aligned}$$

The current mathematical question is no longer how to create the binary threshold inside a safe interval, but how to extend the Lucas predicate beyond the quotient-one regime so that a broad interval might be handled without explicit segmentation.

The current computational question is how to evaluate the required range products more directly, or replace them altogether with a closed-form or sublinear Lucas test.

17 Recorded Experimental Results

The following table records selected outputs without drawing statistical conclusions from them.

Table 1: Selected reported experiment outputs.

Experiment	Scale	Cases	Successes	Probes	GCD calls
138	small	100	100	–	–
140	small	100	100	–	–
145	small	900	900	–	–
152	10^{18}	2	2	2, 3	399453, 621252
152	10^{19}	1	1	5	2175262
153	10^{18}	2	2	88, 85	88, 85
153	10^{19}	1	1	103	103
154	10^{18}	2	2	56, 89	56, 89
154	10^{19}	1	1	91	91
155	10^{18}	2	2	86, 90	86, 90
155	10^{19}	1	1	60	60
156	10^{18}	2	2	87, 86	87, 86
156	10^{19}	1	1	90	90

The table is intentionally descriptive. It reports the recorded experiment outputs rather

than interpreting them statistically.

18 Conclusion

The continuation of the research has produced a concrete Lucas-theoretic localization mechanism for the smaller prime factor of a semiprime. Under a quotient-one condition, the divisibility of a specially chosen binomial coefficient becomes a threshold predicate:

$$p \mid \binom{m}{m-t} \iff p > t.$$

This predicate supports binary left/right localization inside intervals satisfying

$$U < 2L.$$

The experimental implementation subsequently reduced the number of GCD operations to approximately one per binary probe and introduced cumulative products, paired factors, and Montgomery multiplication for the remaining modular arithmetic.

The next mathematical direction is to remove the dependence on quotient-one segmentation by deriving a useful Lucas predicate across multiple quotient regimes. The next computational direction is to replace the explicit product scan by a more direct evaluation of the corresponding factorial or binomial residue.

No claim is made here that the resulting procedure constitutes a practical general-purpose integer factorization algorithm. This document records the mathematical and experimental development of the construction and its current implementation state.